Jarvis Build Plan

Soham · November 5, 2026

Vision and goals

Jarvis is a personal agentic harness that runs 24/7 on the laptop, knows Soham's context, and gets real work done across apps, the web and code, reachable by voice on the laptop and by text or voice from the phone.

- **One front door:** talk or type from laptop or phone; same brain, same memory, same task list.
- **Does work, not just chat:** dispatches coding jobs to Claude Code, Codex and Gemini CLI, researches the web, drives desktop apps and the browser.
- **Knows you:** a persistent profile and memory (projects, people, preferences, schedule) that every agent reads.
- **Safe by default:** risky actions wait for a one-tap approval on the phone.
- **Uses the subscriptions already paid for:** Claude Pro, ChatGPT Pro and Gemini Pro through their official CLIs, so near-zero extra cost.
- **Built to grow:** new skills and tools drop in as plugins or MCP servers without touching the core.

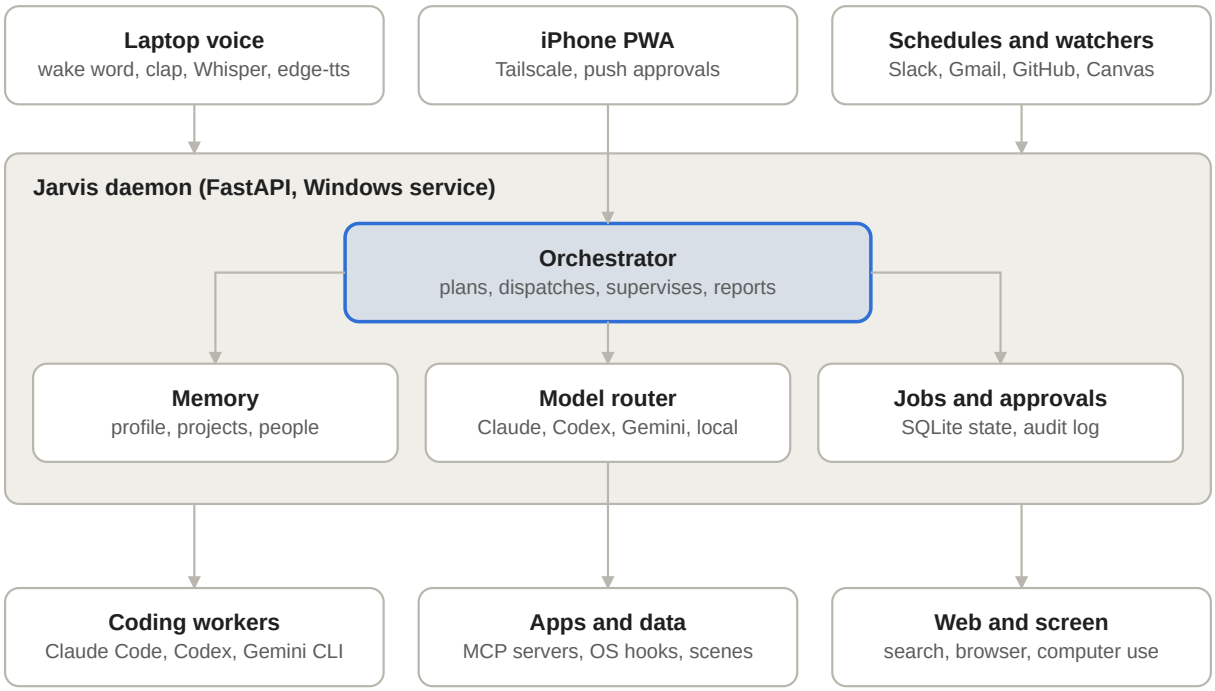
Decisions so far

Topic	Decision
Laptop OS	Windows (service via NSSM or Task Scheduler, PowerShell for OS hooks)
Phone	iPhone (PWA added to Home Screen, push via ntfy iOS app)
Main uses	Coding, school, job search, daily life
First apps	Google suite, dev tools (GitHub, VS Code, terminal), messaging, media
Autonomy	Auto for reading and sandboxed coding; phone approval for sending, pushing, installing
Personality	Classic JARVIS, polite British butler
Voice wake	"Hey Jarvis" wake word plus a hotkey
Uptime	Laptop stays awake while plugged in, including lid closed
Data access	All three providers may see private repos, school and email

Architecture overview

Everything runs through one daemon on the laptop: interfaces feed it requests, the orchestrator plans with memory and the model router, and work fans out to coding agents, app integrations and the web.

One always-on daemon sits between every interface and every worker



Jarvis architecture · 3 interfaces, 1 daemon, 3 worker groups

Interfaces never talk to workers directly; every action passes through the orchestrator and the approval queue.

Model routing across your three subscriptions

Each subscription comes with an official CLI that runs headless, so Jarvis drives the CLIs instead of paying for API keys. A small router picks the worker by task type, and falls back to another provider when one hits its usage window.

Provider	Headless entry point	Best used for	Watch out for
Claude Pro	`claude -p` (Claude Code), Claude Agent SDK	Orchestrator brain, complex coding, planning, tool-heavy work	Pro has rolling usage limits; long agent loops burn through them
ChatGPT Pro	`codex exec` (Codex CLI)	Second coding worker, code review, parallel tasks	Run in a sandbox mode, not full-auto, outside throwaway dirs
Gemini Pro	`gemini -p` (Gemini CLI)	Huge-context reading (whole repos, long PDFs), web-grounded search, cheap summaries	Review Google's data terms for anything sensitive
Local (Ollama)	HTTP on localhost	Wake-word follow-ups, intent routing, offline fallback	Small models only; not for real reasoning

Routing rules for v1:

1. Intent classification and quick replies: local model or Gemini.
2. Anything that edits code or files: Claude Code first, Codex if Claude is rate limited or for a second opinion.
3. "Read all of this" tasks over 200k tokens: Gemini CLI.
4. Cross-checking: for important changes, one worker writes and a different provider reviews.

Agentic harness design

The harness is an orchestrator that never does heavy work itself: it understands the request, plans, hands steps to workers, tracks them, and reports back. Workers coordinate through a shared task store in text, and only Jarvis speaks to you.

Task lifecycle

1. **Intake:** voice or text arrives, gets normalized to text, tagged with channel (laptop voice, phone, scheduled).
2. **Plan:** orchestrator reads memory, writes a short plan with steps, tools and the worker for each.
3. **Gate:** any step in an approval class (see Security) is queued and pushed to the phone.
4. **Dispatch:** each step becomes a job: a CLI worker in a repo, an MCP tool call, a browser task, or a sub-agent.
5. **Watch:** a supervisor streams job logs, detects "waiting for input" or failure, retries or escalates.
6. **Report:** Jarvis summarizes results by voice or push notification and writes what was learned to memory.

Core orchestrator tools

Tool	What it does
`agent_start(task, repo?, worker?)`	Starts a Claude Code, Codex or Gemini job in a git worktree and returns a job id at once
`agent_status` / `agent_result`	Lists running jobs, fetches a job's final output
`agent_send(job, message)`	Answers a worker's question or adds a follow-up
`agent_stop(job)`	Cancels a job
`web_search`, `web_fetch`	Research through a search API or Gemini grounding
`memory_read`, `memory_write`	Personal context store
`approve_request`	Pushes an approval card to the phone and waits
`schedule(task, when)`	Recurring or one-off jobs (morning brief, nightly repo checks)
`run_scene(name)`	Fires a named workspace scene (see Scenes)
MCP servers	Everything else: calendar, email, files, GitHub, Slack, Spotify, desktop control

Design rules

- Every job runs in its own git worktree or scratch dir, so parallel workers never clobber each other.
- Workers inherit your normal Claude Code setup (MCP servers, skills, CLAUDE.md) so terminal and Jarvis behave the same.
- Use Claude Code hooks (PreToolUse, Notification, Stop) to feed events back to the supervisor instead of scraping terminal output.
- All state lives in SQLite (jobs, approvals, transcripts) so a crash or reboot resumes cleanly.

Memory and personal context

Jarvis keeps its knowledge of you as plain Markdown files in a git repo, with a vector index on top for search, so you can read, edit and version everything it believes.

- **Profile:** who you are, school, work, skills, goals (seeded by you in a one-time onboarding chat).
- **Areas:** one file per ongoing project with status and decisions.
- **People:** advisor, teammates, recruiters, with how you know them.
- **Preferences:** how you want answers (short, casual, no em dashes), voice, notification hours.
- **Episodic log:** dated summaries of what Jarvis did each day, rolled up weekly.

How it fills: a background pass after each conversation proposes facts, and only things you stated get saved; you can say "forget X" any time. Retrieval: every plan step pulls the profile plus the top matching files (SQLite FTS + embeddings via a local model), so workers get context without you repeating it.

Integrations: apps, web and the computer

Prefer APIs and MCP servers first, browser automation second, and screen-level computer control only as a last resort, because each step down is slower and more fragile.

Layer	How	Examples
MCP servers	Official or community servers registered once, shared by orchestrator and workers	GitHub, Google Calendar, Gmail, filesystem, Slack, Spotify, Notion
Native OS hooks	PowerShell, pywinauto, Windows notifications	Open apps, set volume, focus and arrange windows, clipboard
Browser agent	Playwright with a dedicated Chrome profile	Sites with no API, forms, dashboards, Canvas
Computer use	Screenshot + click loop (Claude computer use)	Desktop apps with no API at all
Web research	Search API (Brave or Tavily) + fetch, or Gemini CLI with Google grounding	"Find repos like X", news, docs lookup
Coding agents	Claude Code, Codex, Gemini CLI as workers	Build features, fix bugs, review PRs, search GitHub for inspiration

Watchers: connectors that check on their own

Each connector gets a watcher that polls (or receives webhooks) on a schedule, filters what matters, and only pings the phone when something needs you.

Connector	Watcher checks	Pings you when
Slack	DMs, mentions, chosen channels every 5 min	Someone needs a reply; Jarvis drafts it for one-tap send
Gmail	New mail every 10 min	Recruiter, professor or interview email; deadline mentioned
Google Calendar	Next 24h	15 min before events, conflicts, new invites
GitHub	PRs, issues, CI on your repos	CI fails, review requested, PR merged
Canvas	Assignments and announcements	Due within 48h, new grade or announcement
Discord / WhatsApp / Telegram	DMs	Message from a starred contact
Job boards (LinkedIn, Handshake, Greenhouse)	New postings matching saved searches daily	New-grad SWE/AI role matches
Notion / Google Drive	Shared docs	Comment or edit mentioning you

Phone control has two layers: the Jarvis PWA and push for everything, plus Claude Code Remote Control (`claude remote-control`) so a coding job Jarvis started can be opened and steered live from the Claude iOS app.

Interfaces: voice, phone, always on

One daemon serves every interface over the same WebSocket API, so laptop voice, phone and a terminal client all share state.

Laptop voice (fully free, mostly local)

- Wake word: openWakeWord ("hey jarvis" model, about 2% CPU) plus a push-to-talk hotkey.
- Speech to text: faster-whisper locally, with voice activity detection to know when you stop.
- Text to speech: edge-tts (Microsoft neural voices, free, British voice) with Piper as an offline fallback; ElevenLabs optional.
- Barge-in: stop speaking the moment you talk over it.

Phone (iPhone)

- Tailscale on laptop and phone, so the daemon is reachable privately from anywhere with no open ports.
- A PWA served by the daemon: chat, push-to-talk (records audio, laptop transcribes), job list, approval cards.
- Push notifications through ntfy for "job done" and "needs approval".

- Optional later: a Telegram bot as a second channel.

Always on (Windows)

- Runs as a service via NSSM or Task Scheduler at logon; watchdog restarts on crash; health endpoint the phone can ping.
- Power settings: no sleep on AC, lid-close does nothing. When the laptop is off, Jarvis is off; a small always-on box can host the phone gateway later.

Scenes and triggers

Borrowed from the double-clap desk starter: one gesture sets up a whole workspace. Jarvis turns that into named scenes in a YAML file, fired by any trigger, and makes the greeting actually useful.

Scene	Trigger examples	What happens
Home	Double clap, "Jarvis, I'm home", laptop unlock after 6pm	Spotify playlist, Chrome with Claude and Gmail, VS Code on the last repo, spoken brief
Deep work	"Focus mode", phone button	Do Not Disturb, lo-fi, close Discord, open today's repo, timer
Class	Calendar event starts	Open Canvas and lecture notes, mute notifications
Job hunt	"Job mode"	Open tracker, recruiter inbox, today's applications
Shutdown	"Goodnight"	Summarize the day to memory, queue overnight coding jobs, lock screen

Upgrades over the original:

- **Any trigger, same scene:** clap, wake word, hotkey, phone button, calendar event or Windows unlock all call `run_scene(name)`.
- **Smarter clap detection:** two spikes in a time window plus a pattern match, so a door slam doesn't fire it; per-room calibration.
- **Greeting with substance:** a 20-second spoken brief from calendar, due dates, job emails and overnight job results.
- **Free voice:** edge-tts British voice by default, ElevenLabs optional.
- **Windows-native app control:** pywinauto and PowerShell to launch, focus, fullscreen and arrange windows per monitor.
- **Re-runnable and editable:** scenes are idempotent and live in `scenes.yaml`; say "add YouTube to Home" and Jarvis edits it.

Security and safety

Anything that can run code on your laptop from your phone is a remote shell, so the harness never skips permissions and every risky action needs your tap.

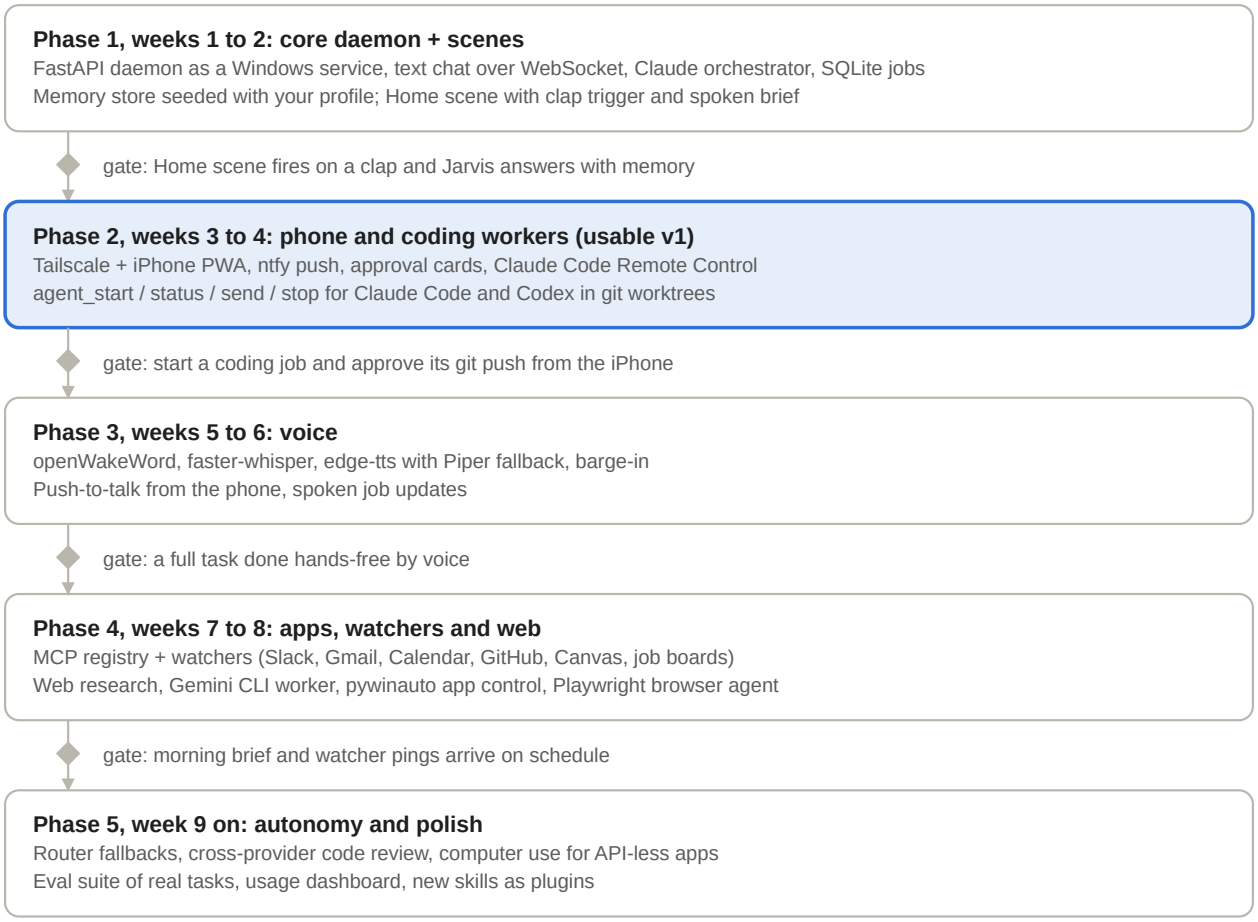
Action class	Policy
Read files, search web, read calendar/email	Auto
Edit code inside a job's worktree	Auto (Claude Code `acceptEdits` or `auto` mode, never `--dangerously-skip-permissions`)
Shell commands outside the worktree, installs, git push	Ask on phone
Send email/messages, post anything, calendar invites	Ask on phone, show the exact text
Purchases, payments, logins, deleting data	Never automatic; Jarvis prepares, you do it

- **Network:** daemon binds to localhost and the Tailscale interface only; phone auth via Tailscale identity plus a device token.
- **Prompt injection:** web pages, emails and repo files are data, never instructions; anything they "ask" for goes to the approval queue.
- **Secrets:** tokens in Windows Credential Manager, never in memory files or prompts.
- **Data boundaries:** work material stays off personal Jarvis entirely; passwords and tokens are never sent to models.
- **Audit log:** every tool call and approval recorded in SQLite, viewable from the phone.

Phased roadmap

Ship something you use daily by week 4, then layer voice, apps and autonomy on top; each phase ends with a gate you can actually test.

Usable from the phone by week 4, hands-free by week 6



Jarvis roadmap · 5 phases, 4 gates

Weeks assume part-time work alongside classes; phases 3 and 4 can swap if app integrations matter more than voice.

Tech stack, repo layout, open questions

Stack: Python 3.12, FastAPI + WebSockets, SQLite, Claude Agent SDK for the orchestrator, asyncio subprocess for CLI workers, faster-whisper, openWakeWord, edge-tts, Piper, pywinauto, Playwright, Next.js PWA, Tailscale, ntfy.

```
jarvis/
  core/          orchestrator, planner, router, supervisor
  workers/       claude_code.py, codex.py, gemini.py, browser.py
  tools/         agent_*, web, memory, approvals, schedule, scenes
  watchers/      slack, gmail, calendar, github, canvas, jobs
  mcp/           mcp server registry and configs
  memory/        markdown store + index
  voice/         wake, clap, stt, tts, audio loop
  api/           FastAPI app, websocket, auth
  web/           Next.js PWA for phone
  service/       Windows service setup (NSSM / Task Scheduler)
  scenes.yaml
  tests/
```

Open questions

- Does the Agent SDK run on your Claude Pro login, or should the orchestrator call `claude -p` directly to stay on the subscription?
- Is Claude Code Remote Control available on your plan?
- Want the phone to work when the laptop is asleep (needs a small always-on gateway)?